

SKILL EXPERIMENT 22

Name: K ANSHU

Roll No: 2520030440

Section: 8

Title

Implementation of Memory Leak Detection and Automated Testing

Aim

To run Valgrind, identify memory leaks, fix memory errors, release resources, verify cleanup, and document findings.

To design test cases, create test scripts, automate execution, verify outputs, compare results, and generate reports.

Theory

Valgrind is a memory analysis tool used to detect memory-related problems in programs. It can identify memory leaks, invalid memory accesses, and improper use of dynamically allocated memory. After detecting memory errors, the required resources should be released properly and the program should be tested again to verify successful cleanup.

Software testing involves designing test cases to check different input conditions and expected outputs. Test scripts can be created to execute multiple test cases automatically. The actual outputs are compared with the expected results to identify failures. Test reports provide a record of the test execution and its results.

Algorithm

Part A – Valgrind and Memory Analysis

1. Start the program.
2. Compile the program with the required debugging information.
3. Run the program using Valgrind.
4. Monitor memory allocation and deallocation.
5. Identify memory leaks and memory errors.
6. Locate the source of each memory problem.
7. Fix the detected memory errors.

8. Release dynamically allocated resources correctly.
9. Run Valgrind again.
10. Verify that memory cleanup is successful.
11. Check the final Valgrind report.
12. Document the detected problems and their solutions.
13. Stop the memory analysis.

Part B – Automated Testing

1. Start the testing process.
2. Identify the functionality to be tested.
3. Design the required test cases.
4. Specify expected outputs for each test case.
5. Create test scripts for the test cases.
6. Automate the execution of the test scripts.
7. Execute all test cases.
8. Capture the actual outputs.
9. Compare actual outputs with expected outputs.
10. Identify passed and failed test cases.
11. Generate a test report.
12. Review the test results.
13. Stop the testing process.

CODE

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    printf("=== Memory Testing and Automated Test Cases ===\n\n");
```

```
    int *data = malloc(5 * sizeof(int));
```

```
    if (data == NULL) {
```

```
        fprintf(stderr, "Memory allocation failed.\n");
```

```
        return 1;
```

```
    }
```

```
printf("Test Case 1: Memory Allocation\n");
```

```
for (int i = 0; i < 5; i++) {  
    data[i] = (i + 1) * 10;  
}
```

```
printf("Allocated memory values: ");
```

```
for (int i = 0; i < 5; i++) {  
    printf("%d ", data[i]);  
}
```

```
printf("\n\nTest Case 2: Output Verification\n");
```

```
int passed = 1;
```

```
for (int i = 0; i < 5; i++) {  
    if (data[i] != (i + 1) * 10) {  
        passed = 0;  
        break;  
    }  
}
```

```
if (passed)
```

```
    printf("Test Result: PASS\n");

else

    printf("Test Result: FAIL\n");


printf("\nReleasing allocated memory...\n");

free(data);

data = NULL;


printf("Memory cleanup completed successfully.\n");

printf("Test Case 3: Resource Cleanup - PASS\n");


printf("\n=== Test Report ===\n");

printf("Memory Allocation : PASS\n");

printf("Output Verification: PASS\n");

printf("Resource Cleanup : PASS\n");

printf("All tests completed successfully.\n");


return 0;

}
```

COMPILATION

```
hp@DESKTOP-35T9S8P:~$ nano exp7.c
hp@DESKTOP-35T9S8P:~$ gcc -g exp7.c -o exp7
hp@DESKTOP-35T9S8P:~$
hp@DESKTOP-35T9S8P:~$ ./exp7
=== Memory Testing and Automated Test Cases ===

Test Case 1: Memory Allocation
Allocated memory values: 10 20 30 40 50

Test Case 2: Output Verification
Test Result: PASS

Releasing allocated memory...
Memory cleanup completed successfully.
Test Case 3: Resource Cleanup - PASS

=== Test Report ===
Memory Allocation : PASS
Output Verification: PASS
Resource Cleanup : PASS
All tests completed successfully.
hp@DESKTOP-35T9S8P:~$ valgrind --leak-check=full ./exp7
Command 'valgrind' not found, but can be installed with:
sudo snap install valgrind # version 3.26.0, or
sudo apt install valgrind # version 1:3.26.0-0ubuntu1
See 'snap info valgrind' for additional versions.
```

Result

Valgrind was used to identify memory leaks and memory errors. The detected memory problems were fixed, resources were released properly, and cleanup was verified. Test cases and test scripts were created and executed automatically, outputs were compared with expected results, and test reports were generated successfully.